



Fondamenti di Analisi dei Dati

from **data analysis** to **predictive techniques**

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Studi in Informatica

Dip. di Matematica e Informatica

Università di Catania



Università
di Catania

Density Estimation

Understanding the probability distribution of data unlocks powerful insights—from discovering customer segments to detecting anomalies and generating new content.

What is Density Estimation?

We have seen that clustering algorithms aim to mine the underlying structure of data by breaking it into coherent groups.

An alternative **smoother** approach would be to study the probability

$$P(X)$$

where X is the data. Indeed, if we could determine the probability value of an given point in space , we could naturally discover clusters in the data (**zones with high probability values**).

In practice, if variable X is continuous, we aim to find the density function:

$$f(x)$$

Knowing the distribution of the data is also useful for a number of things:



Exploratory Data Analysis

Understand the shape, modality, and skewness of data distributions for deeper insights into underlying patterns.



Anomaly Detection

Identify unusual data points or events by modeling typical behavior, useful for fraud detection and quality control.



Clustering & Classification

Discover high-density regions to group similar data points, informing decision boundaries in machine learning models.



Generative Models

Learn complex data distributions to synthesize new, realistic samples for classification, data augmentation or content generation.

Density estimation is fundamental for understanding complex data distributions, uncovering intrinsic patterns, and enabling advanced statistical modeling and machine learning techniques.

Two Approaches to Density Estimation

Non-Parametric Methods

Estimate density directly from data without strong assumptions about the source distribution.

Advantages: Few hyperparameters, no assumptions needed, flexible.

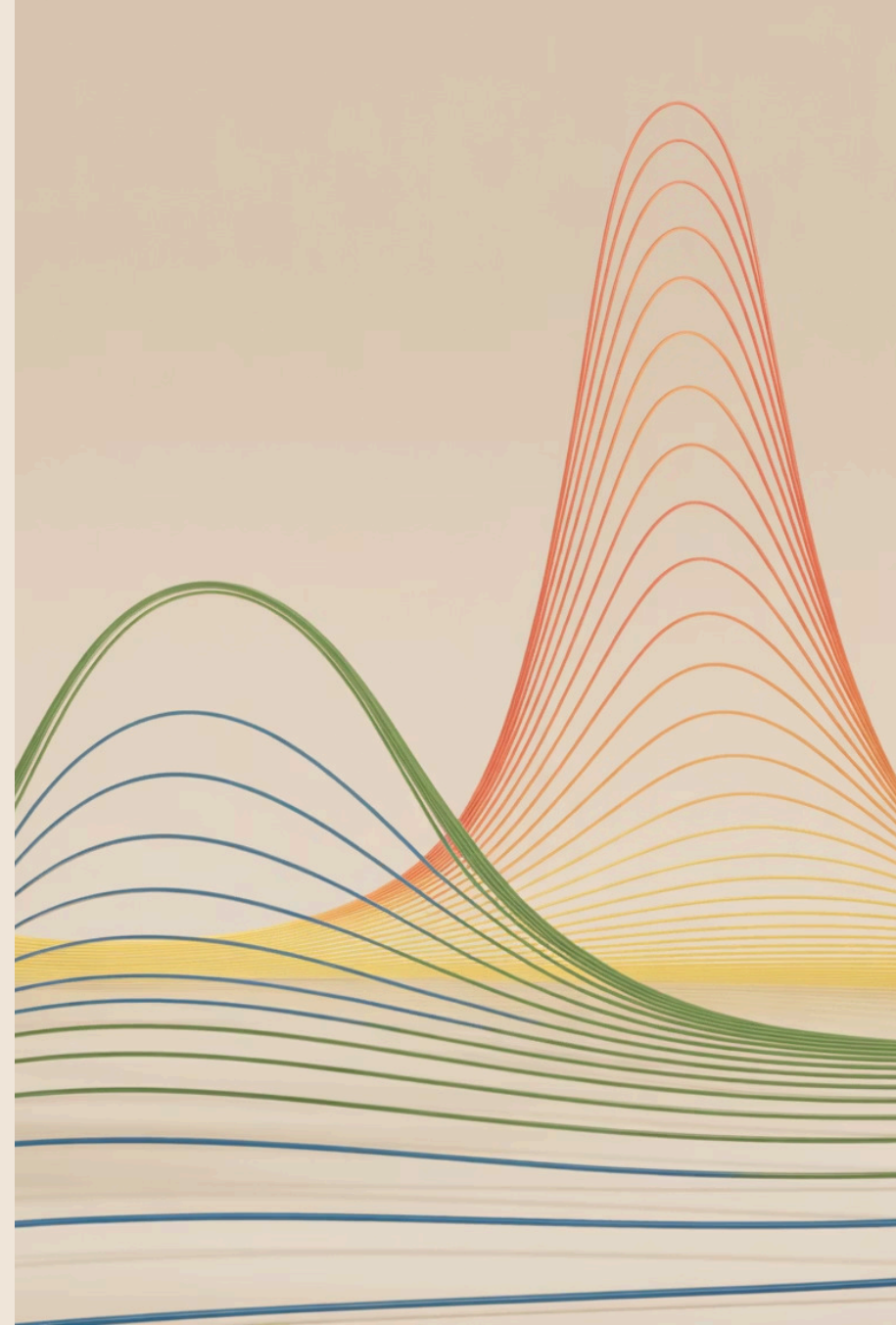
Disadvantages: No analytical form for $f(x)$, computationally intensive.

Parametric Methods

Fit a known distribution (like a Gaussian) to the data.

Advantages: Analytical form, compact representation, interpretable.

Disadvantages: Strong assumptions, requires optimization, may not fit complex data.



Non-Parametric Approaches

We will see two kinds of non-parametric approaches.

1

Fixed Windows

Discretize feature space into a fixed grid of bins and count points in each bin—the multidimensional histogram approach.

2

Mobile Windows

Center a "window" on every data point and sum their influence—creates smoother estimates using Kernel Density Estimation.

Both methods estimate density directly from data, but mobile windows (KDE) produce significantly smoother and more accurate results.

Fixed Windows: The Histogram Method

The simplest approach extends 1D histograms to multiple dimensions by creating a grid of bins (or regions) R_i . Recall that in order for f to be a valid density, it must satisfy:

$$\int f(\mathbf{x}) d\mathbf{x} = 1$$

To achieve this, we define the density of a bin as the probability of the bin (measured as the number of elements in the bin), divided by the volume of the bin:

$$f(\mathbf{x} \in R_i) = \frac{P(R_i)}{V(R_i)} = \frac{|R_i|/N}{V(R_i)}$$

For instance, if the bin is a 2×2 cell, then the "volume" will be 4.

Note that this is coherent with physics, where we define density as mass divided by volume: $\rho = \frac{m}{V}$

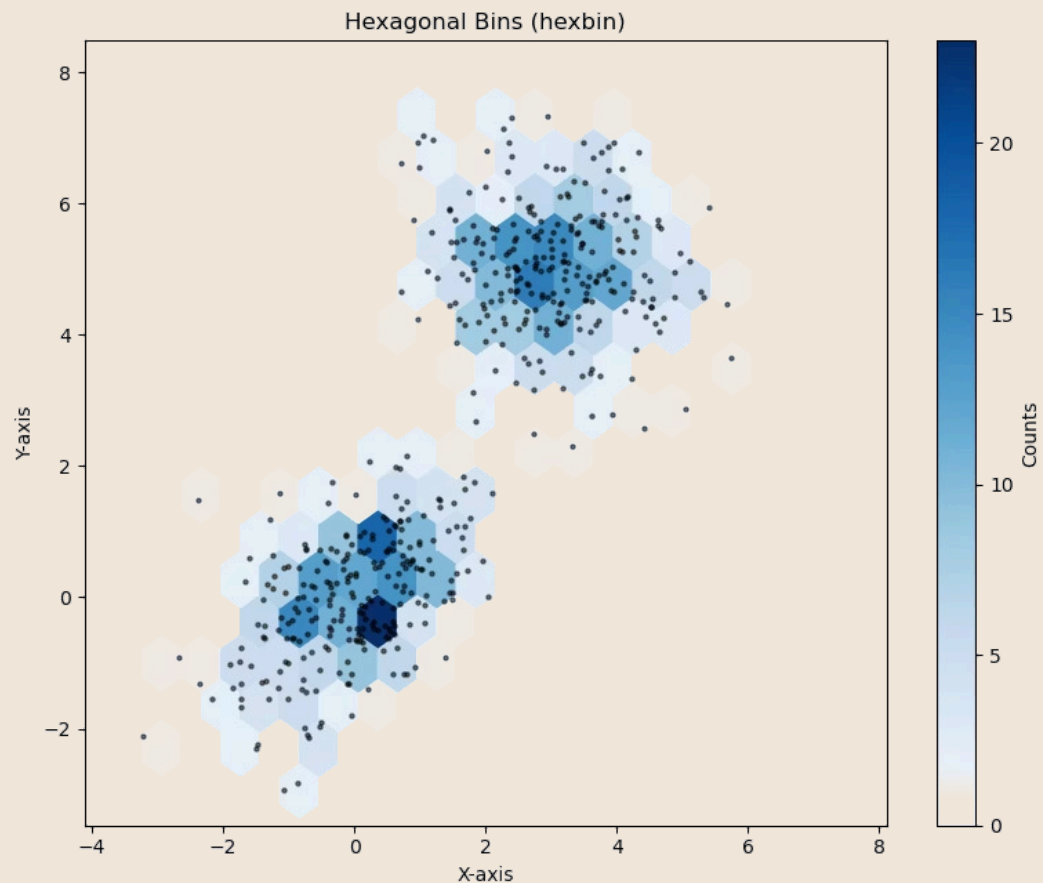
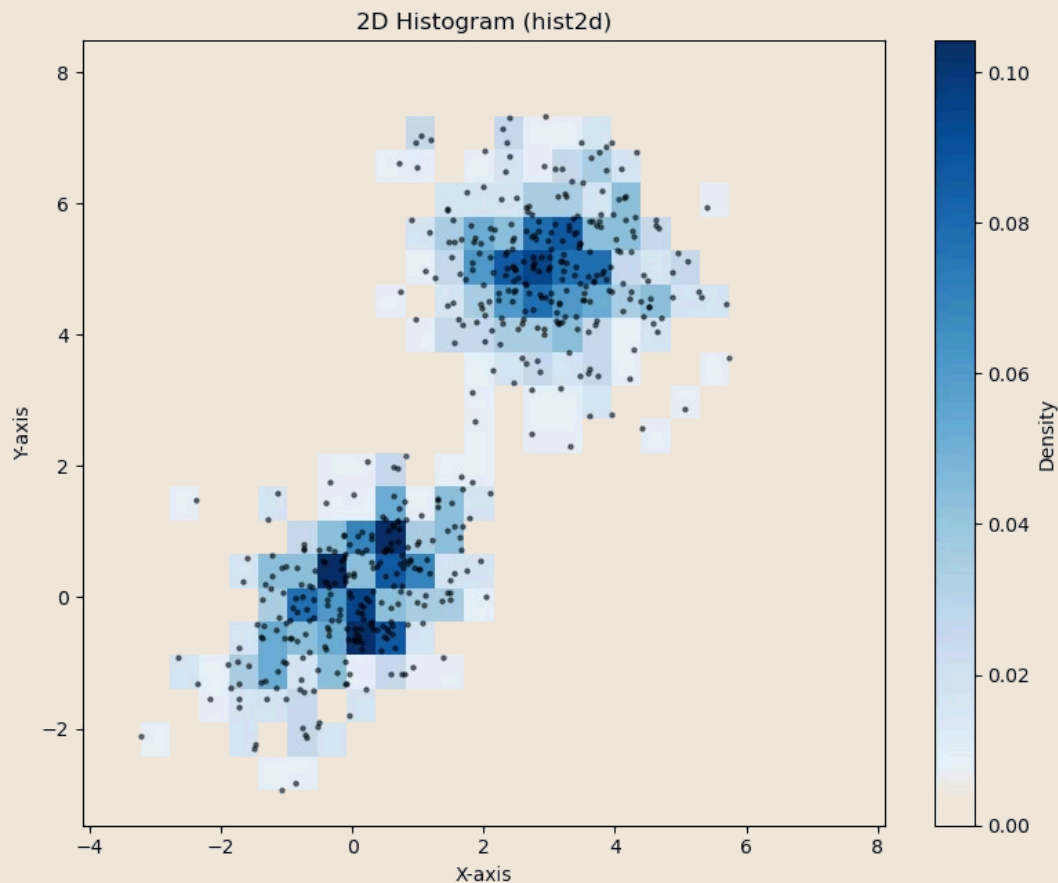
This normalization ensures that the area will be equal to 1:

$$\int f(\mathbf{x}) d\mathbf{x} = P(\mathbf{x} \in \cup_i R_i) = \sum_i P(R_i) = \sum_i f(\mathbf{x} \in R_i) V(R_i) = \sum_i \frac{|R_i|/N}{V(R_i)} V(R_i) = 1$$

This normalization ensures that a small, crowded bin has high density while a large, sparse bin has low density—even with the same point count. The total volume integrates to 1, satisfying the requirements of a probability density function.

Histogram Variants: Square vs. Hexagonal Bins

Fixed Window Density Estimation



Hexagonal binning tiles space more naturally, often producing smoother visual results.

Limitations of Histograms

Arbitrary Boundaries

Bin edges are placed without regard to data structure, causing identical datasets to produce different densities based solely on grid position.

Blocky Results

The resulting density estimate is discontinuous and "staircase-like," failing to capture the smooth nature of true probability densities.

Dimensionality Curse

While generalizable to N dimensions, the curse of dimensionality makes fixed bins increasingly impractical as dimensions grow.

These limitations motivate a more sophisticated approach: mobile windows that adapt to data rather than imposing a rigid grid.



Kernel Density Estimation (KDE)

KDE solves the fixed window problems by placing a "kernel" at each point in space and asking: "How much data is inside this window?" This mobile window approach, also called the Parzen Window method, creates smooth, flexible density estimates.

01

Center a Window

Place a window of radius h at point x

02

Count Nearby Points

Calculate $N(x, h)$ —the number of data points within the window

03

Compute Density

Density = (Probability in window) / (Volume of window)

The Naive Circular Neighborhood

The circular kernel centers a hypersphere of radius h at point $\mathbf{x}$ and counts the number of points inside:

$$f(\mathbf{x}) = \frac{|N(\mathbf{x}, h)|/N}{V(h)}$$

where

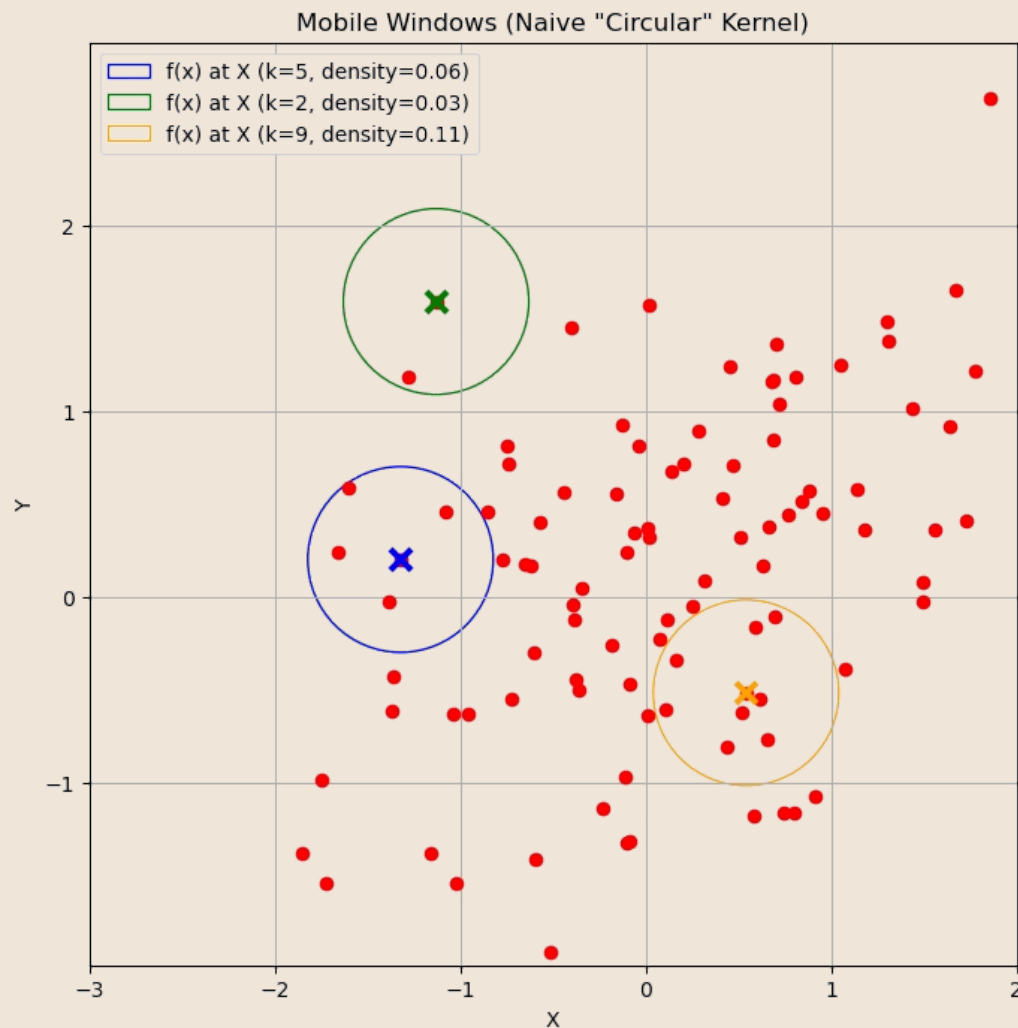
$$N(\mathbf{x}, h) = \{\mathbf{x}' \in \mathbf{X} \text{ s.t. } \|\mathbf{x}' - \mathbf{x}\|_2 \leq h\}$$

is a neighborhood centered at $\mathbf{x}$ with radius h , i.e., containing all points within distance h , and $V(h)$ is the volume of the hypersphere of radius h . For instance, in 2D $V(h) = \pi h^2$.

It can be shown that:

$$\int_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x}) d\mathbf{x} = 1$$

The Problem: Hard decisions create sensitivity. A point is either 100% "in" or 100% "out"—moving $\mathbf{x}$ slightly can cause density to jump abruptly.



This binary decision-making produces non-smooth, "bumpy" estimates. We need kernels that assign decreasing weight as distance increases.

Kernel View

We can express density estimation as a sum over all data points, each weighted by a kernel function K_k :

$$f(\mathbf{x}) = \frac{1}{|\mathbf{X}|} \sum_{i=1}^{|\mathbf{X}|} K_h(\mathbf{x}_i - \mathbf{x})$$

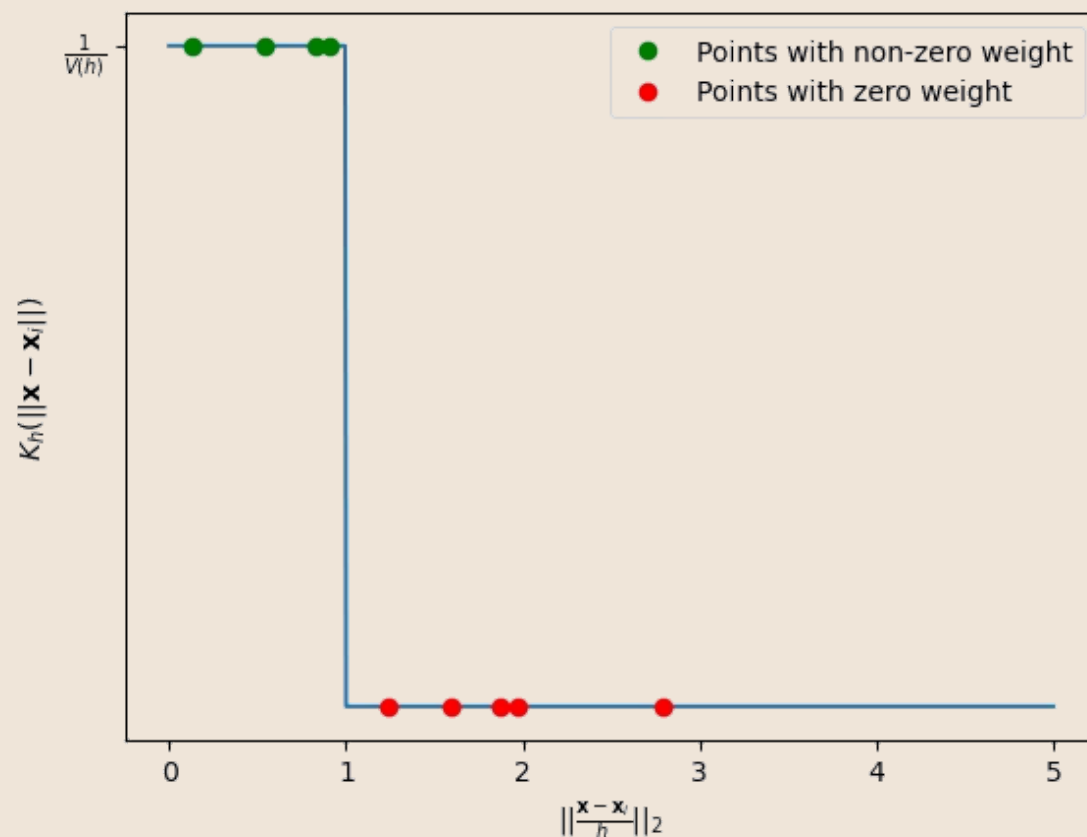
A kernel function takes as input the difference of two points and assigns it a "weight", telling us how much that specific point contributes to the computation of density. We expect that points which are closer to $\mathbf{x}$ contribute more than other.

The circular kernel makes hard decisions, assigning uniform weight $\frac{1}{V(h)}$ inside radius h and zero outside:

$$K_h(\mathbf{x}_i - \mathbf{x}) = \begin{cases} \frac{1}{V(h)} & \text{if } \left\| \frac{\mathbf{x}_i - \mathbf{x}}{h} \right\|_2 \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

This creates discontinuities.

In the plot on the right, points with nearly identical distances (e.g., just a bit further from $\mathbf{x}$ than h) receive vastly different weights. Even small shifts in $\mathbf{x}$ cause green points (non-zero weight) to become red points (zero weight), making the process highly unstable.



The Solution: Smooth Kernels

Smooth kernels eliminate hard boundaries by assigning gradually decreasing weights to points as distance increases. The **Gaussian kernel** is a popular choice:

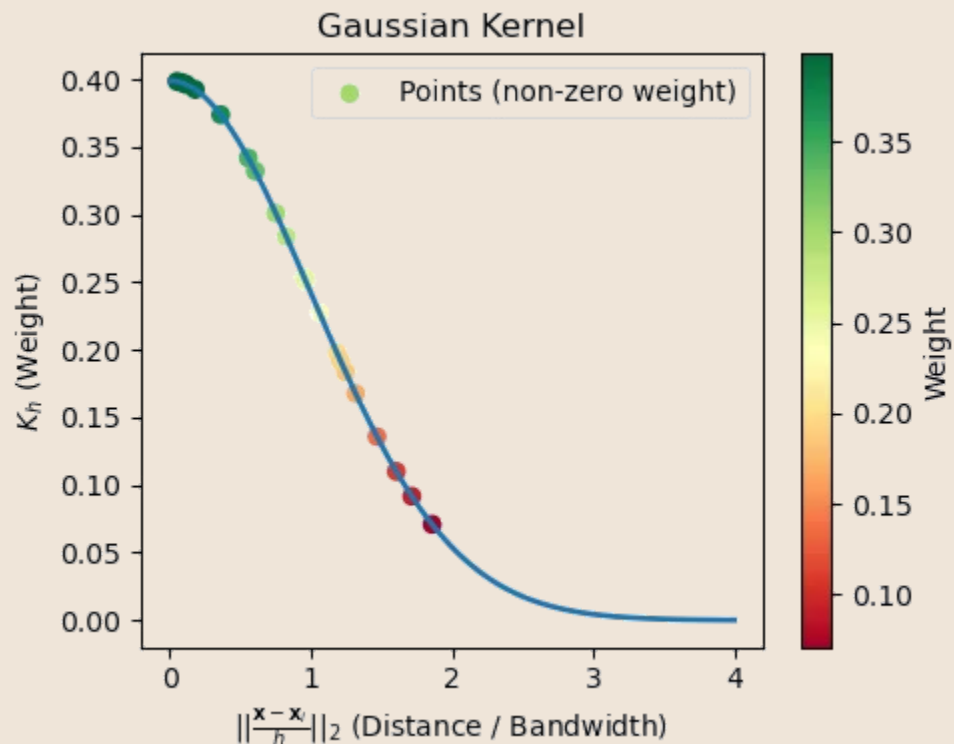
$$K_h(\mathbf{x}_i - \mathbf{x}) \propto \exp\left(-\frac{|\mathbf{x}_i - \mathbf{x}|^2}{2h^2}\right)$$

With a smooth kernel, all points contribute to the density at $\mathbf{x}$, but nearby points contribute far more than distant ones.

The general KDE formula is still a sum of smooth kernels, one centered at each data point:

$$f(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^N K_h(\mathbf{x}_i - \mathbf{x})$$

where $N = |\mathbf{X}|$. This approach produces continuous density estimates that capture the structure of data.



When Efficiency Meets Smoothness: The Epanechnikov Kernel

While the Gaussian kernel solves the sensitivity problem, it **requires computing expensive exponential terms over the entire dataset for every density estimate**. For large datasets, this can become computationally prohibitive.

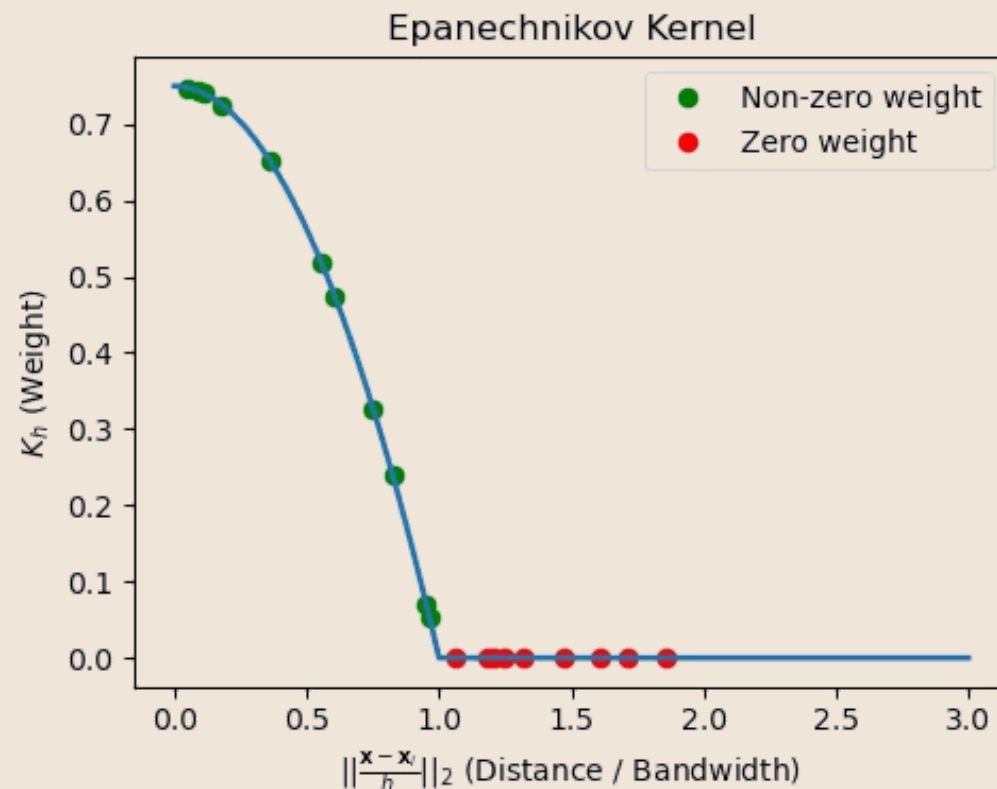
The **Epanechnikov kernel** offers an elegant compromise: smoothness with computational efficiency. It sets weights to zero for points beyond distance h , dramatically reducing computation:

$$K_h(\mathbf{x}_i - \mathbf{x}) = \frac{3}{4h^2} \left(1 - \frac{|\mathbf{x}_i - \mathbf{x}|^2}{h^2} \right) \mathbb{I}(|\mathbf{x}_i - \mathbf{x}| \leq h)$$

Where $\mathbb{I}$ is the indicator function defined as:

$$\mathbb{I}(x) = \begin{cases} 1 & \text{if } x \text{ is true} \\ 0 & \text{otherwise} \end{cases}$$

The quadratic profile maintains smoothness while the compact support (indicator function $\mathbb{I}$) ensures only nearby points are considered, offering the best of both worlds.

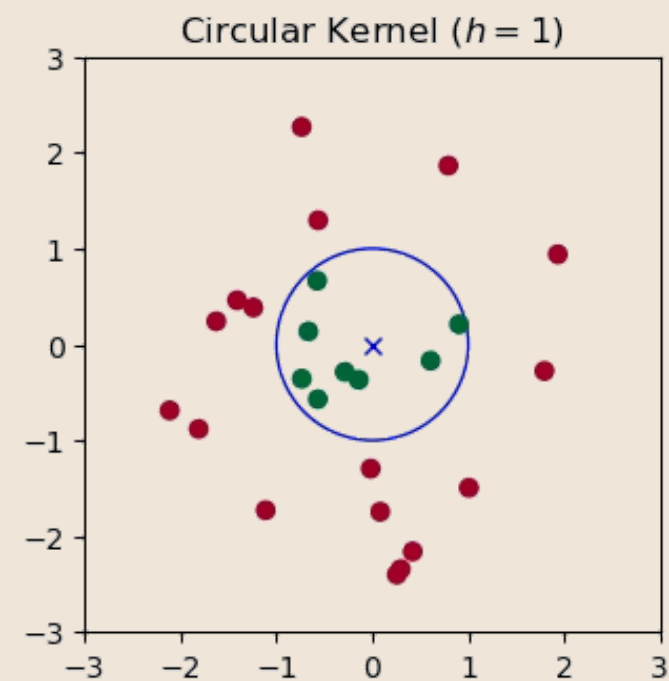
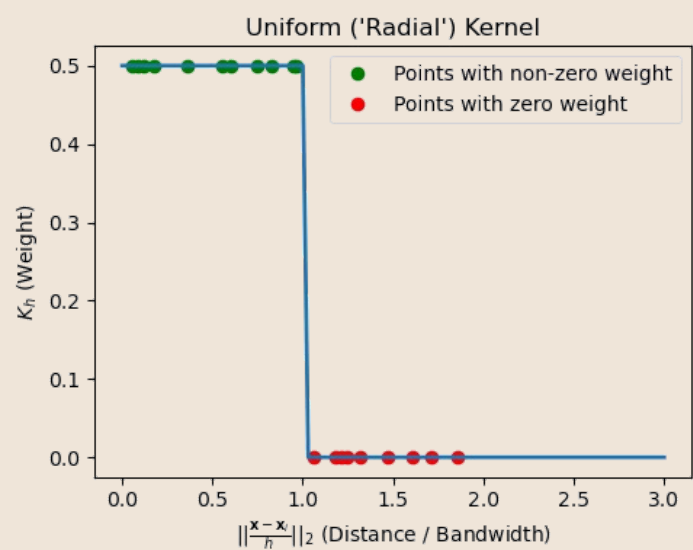


Kernel Comparison

01

Uniform (Circular) Kernel

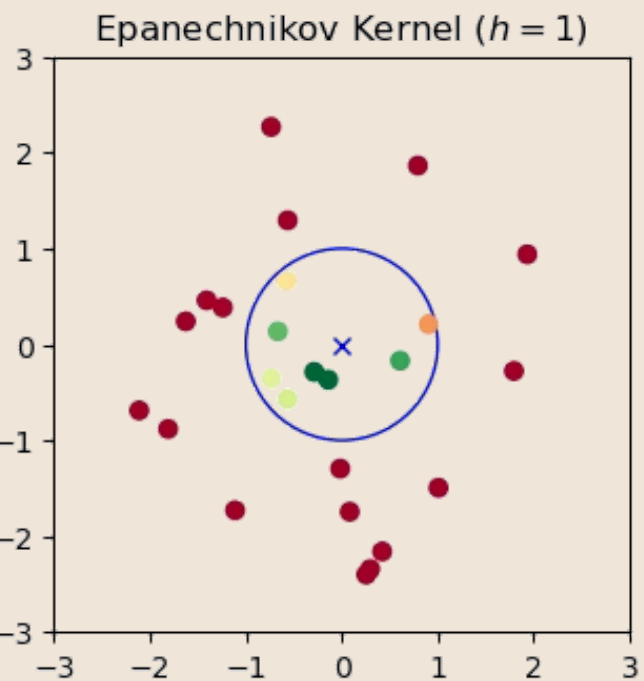
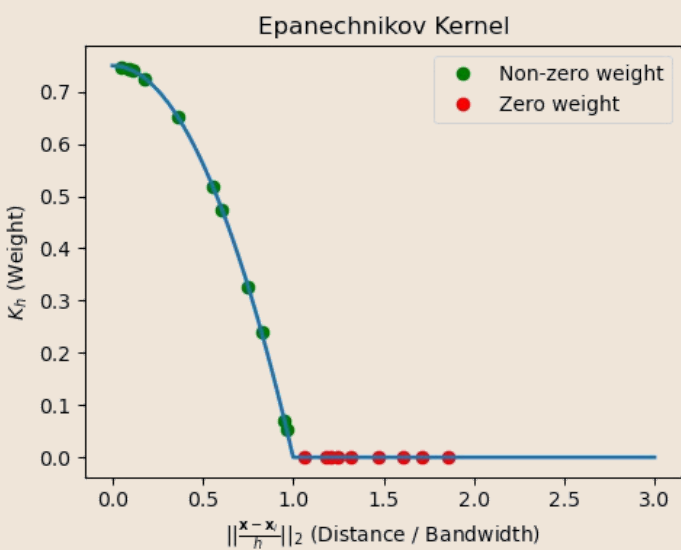
Binary weighting creates visible boundaries. Points inside radius h have equal weight, those outside have zero. Results in non-smooth estimates with artificial discontinuities.



02

Epanechnikov Kernel

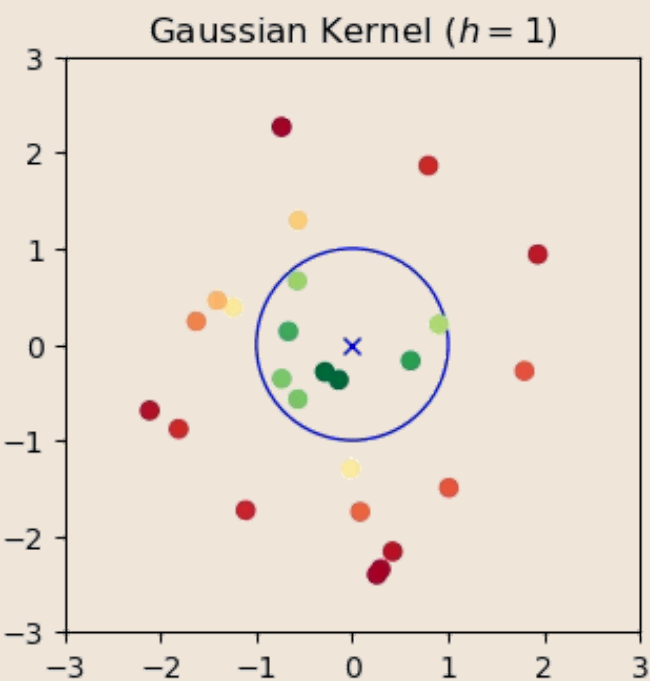
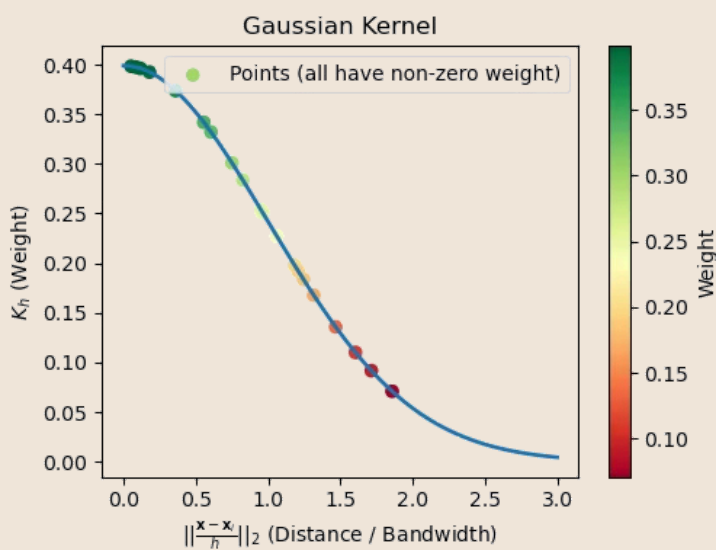
Quadratic decay provides smoothness while compact support limits computation. Weight decreases parabolically from center to radius h , then drops to zero. Optimal in minimizing mean integrated squared error.



03

Gaussian Kernel

Infinite support with exponential decay. All points receive non-zero weight, creating the smoothest estimates. Computationally expensive but theoretically ideal for capturing density structure.



The Bandwidth Dilemma: Bias-Variance Tradeoff

The bandwidth parameter h is **the single most important choice** in density estimation, controlling the fundamental bias-variance tradeoff. Kernel shape matters far less than choosing h correctly.

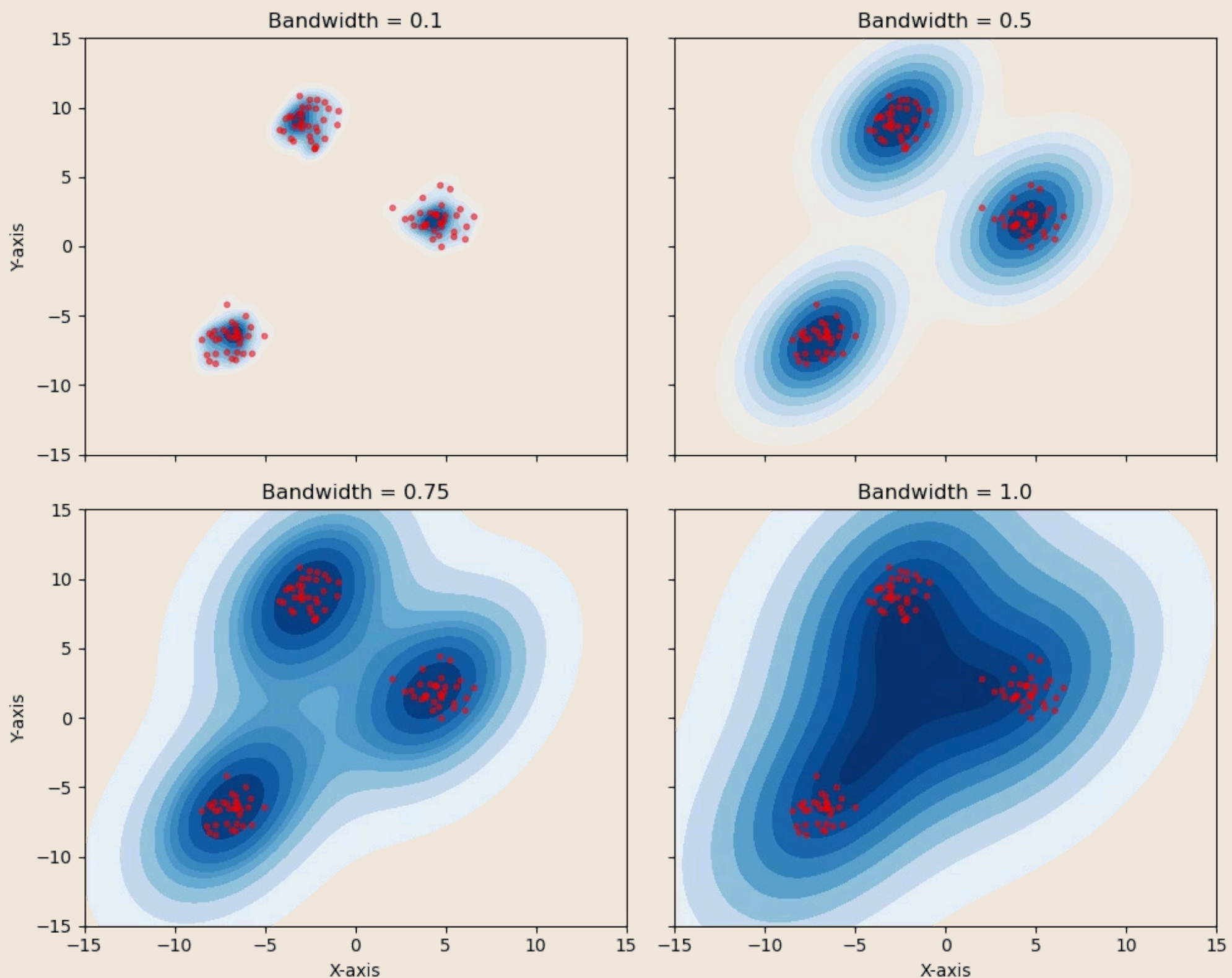
Small Bandwidth ($h = 0.1$)

- **Low Bias, High Variance**
- Trusts only tiny local regions
- Spiky, noisy estimates
- Overfits to training data
- Reveals individual points but misses structure

Large Bandwidth ($h = 1.0$)

- **High Bias, Low Variance**
- Averages over huge areas
- Over-smoothed estimates
- Underfits the data
- Hides true structure like separate clusters

The sweet spot ($h = 0.5$ to 0.75 in many cases) balances local fidelity with global smoothness, capturing genuine structure without noise.



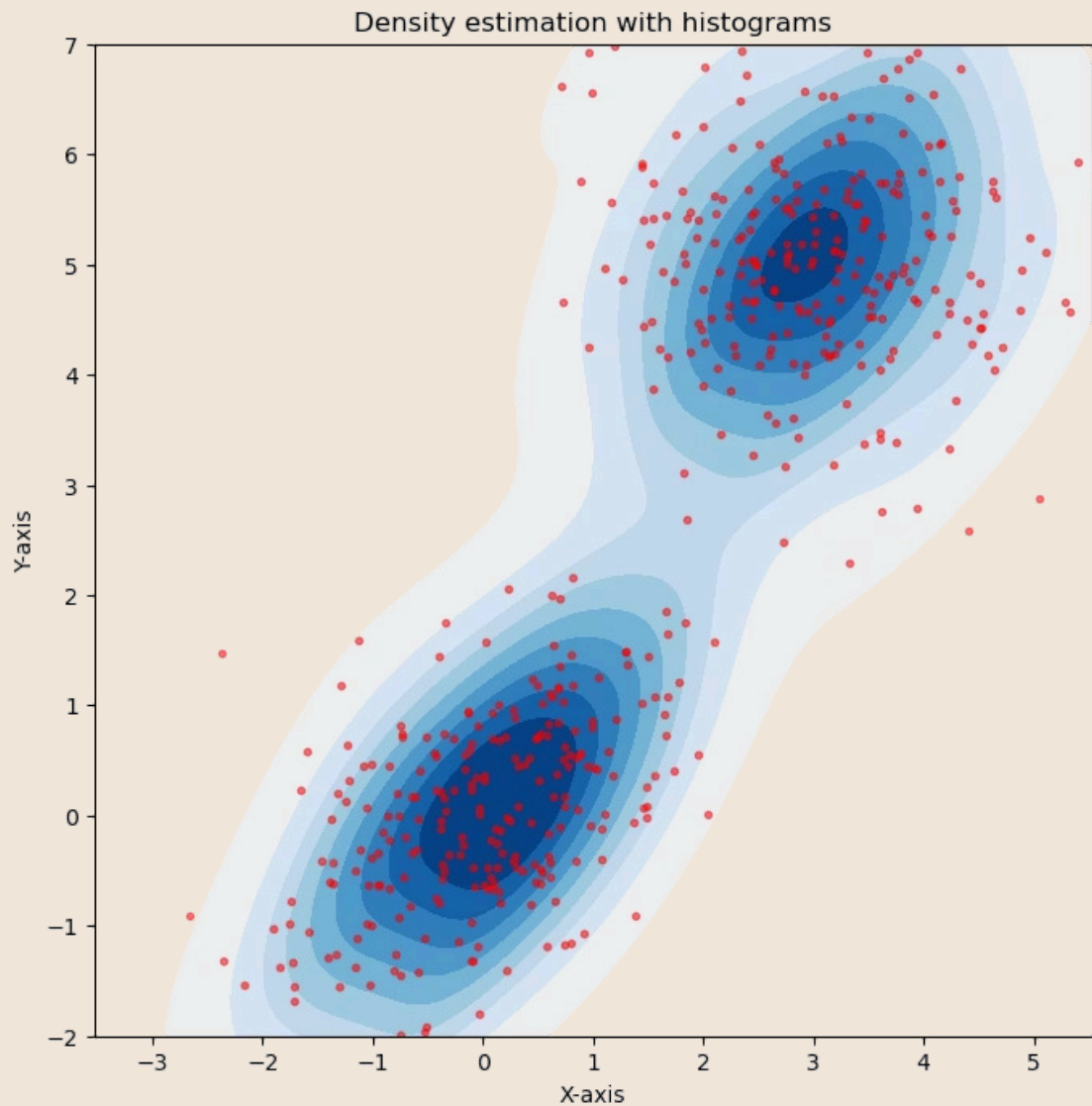
KDE in Action

Using KDE in Python is simple. Here is an example using `seaborn`, but we can also make similar plots directly from `Pandas` or with dedicated `scipy` functions.

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sns.kdeplot(x=data[:, 0],
            y=data[:, 1],
            cmap='Blues',
            fill=True)
```

```
# Display the plot
plt.show()
```



Kernel Density Estimation - Live Demo

Kernel Density Estimation Live Demo

KDE Visualizer

Interactive Kernel Density Estimation

Kernel Function

Epanechnikov

Parabolic shape. Mean square error optimal.



Bandwidth (h) 71



Controls the smoothness.

Render Quality High (Slow)

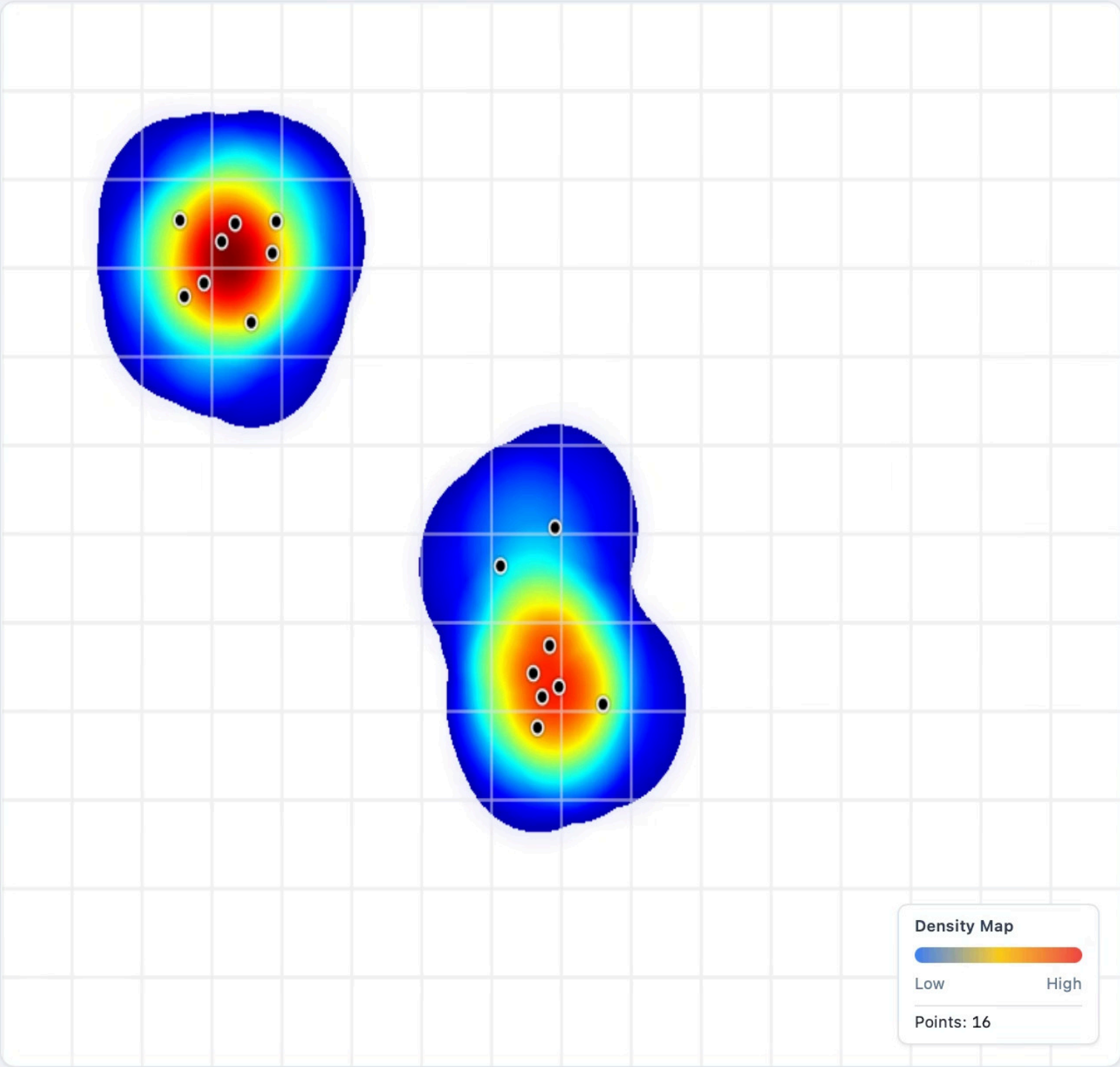


☒ Show Data Points

 Clear  Random

How it works

Kernel Density Estimation sums "bumps" (kernels) placed at each data point. The bandwidth determines the width of the bump. Wider bandwidths oversmooth the data (high bias), while narrow ones capture noise (high variance).



Parametric Density Estimation

Philosophy and Tradeoffs

Advantages

- **Analytical Formulation:** Known equations enable mathematical reasoning and optimization
- **Compact Representation:** Store only parameters, not entire dataset
- **Interpretability:** Parameters reveal data properties (e.g., mean = highest density point)
- **Data Efficiency:** Strong assumptions work better with limited data

Disadvantages

- **Optimization Required:** Finding parameters can be computationally expensive
- **Strong Assumptions:** Model must match true data distribution
- **Potential Mismatch:** Inaccurate if assumptions are violated
- **Limited Flexibility:** Cannot capture arbitrary distributions

Fitting a Gaussian: Maximum Likelihood Estimation

The most common parametric method assumes data follows a **Multivariate Gaussian Distribution**, defined entirely by mean vector μ and covariance matrix Σ :

$$N(\mathbf{x}; \mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^d \det(\Sigma)}} e^{-\frac{1}{2}(\mathbf{x}-\mu)^T \Sigma^{-1}(\mathbf{x}-\mu)}$$

Maximum Likelihood Estimation (MLE) finds parameters that maximize the probability our Gaussian would assign to observed data. We define likelihood as follows:

$$P(\mathbf{X}|\mu, \Sigma) = \prod_i \mathcal{N}(\mathbf{x}_i|\mu, \Sigma)$$

We would like to find the μ and Σ values **maximizing** this expression. To make it more tractable, we assume that all examples $\mathbf{x}_i$ are independent and identically distributed and **minimize** log-likelihood instead:

$$\log P(\mathbf{X}|\mu, \Sigma) = \sum_{i=1}^N \log \mathcal{N}(\mathbf{x}_i|\mu, \Sigma)$$

MLE Solution: Elegantly Simple

Remarkably, the optimal parameters have closed-form solutions: the sample mean and sample covariance matrix:

The Sample Mean

The ML estimate for μ is simply the average of all data points:

$$\mu_{ML} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i$$

This intuitive result says the "most likely" center is exactly where the data is centered on average.

The Sample Covariance

The ML estimate for Σ captures the spread and correlation structure:

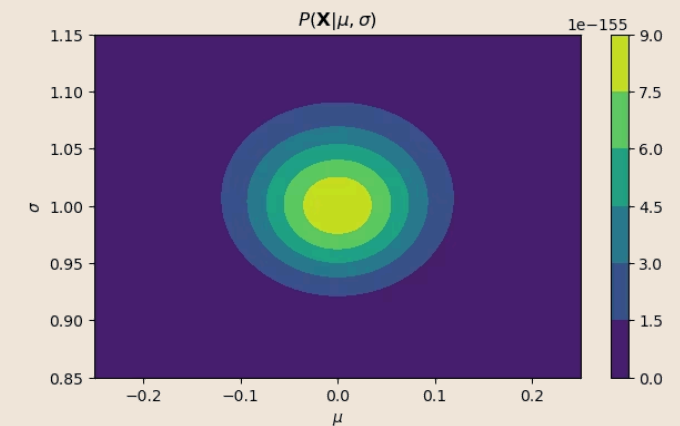
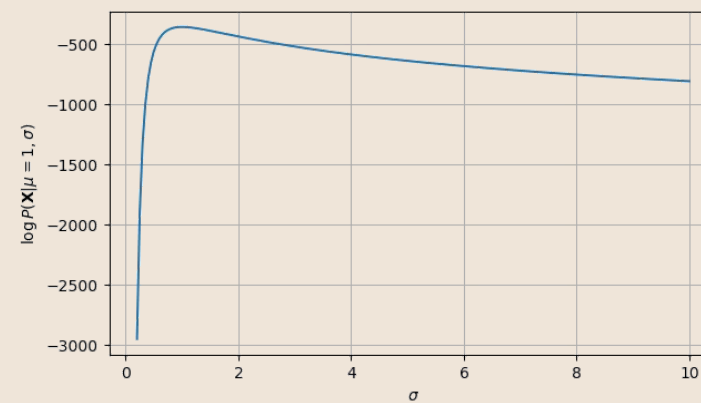
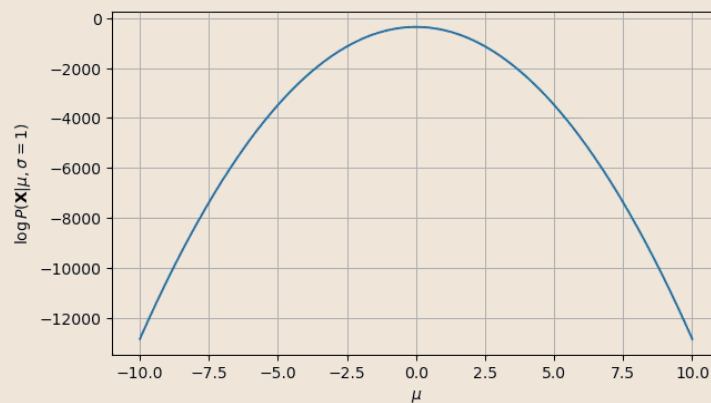
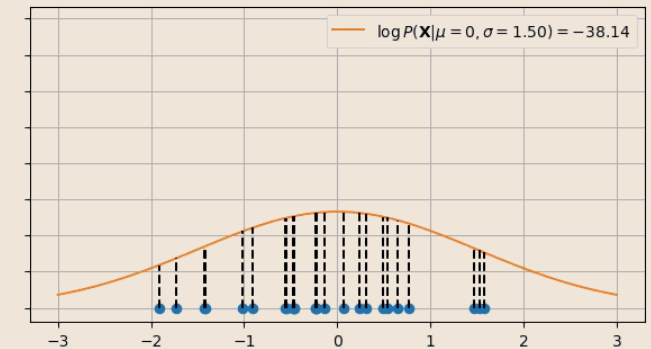
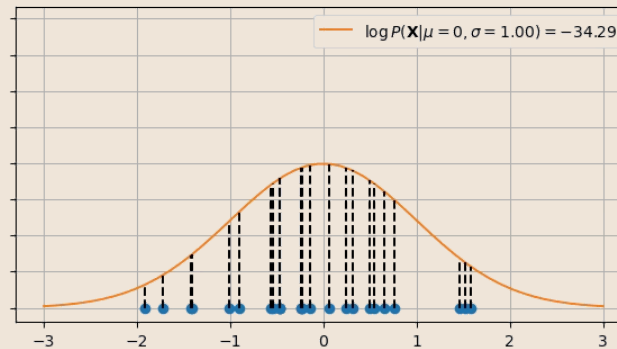
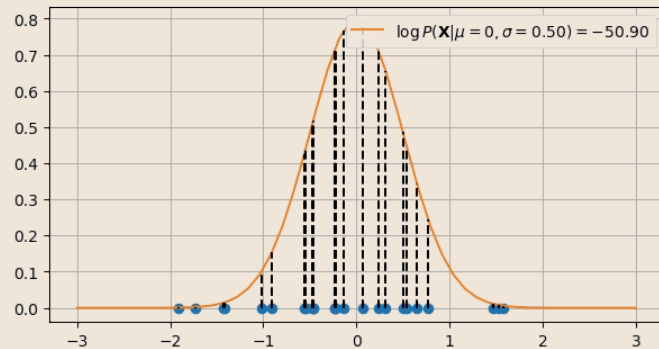
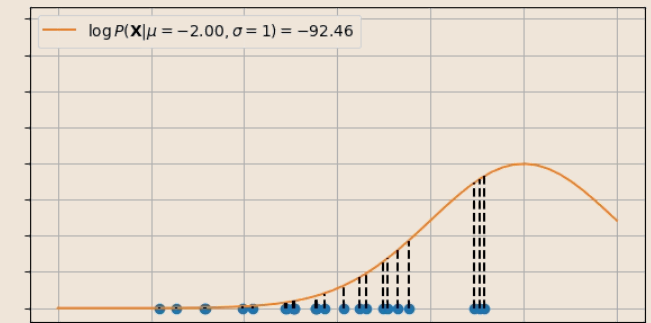
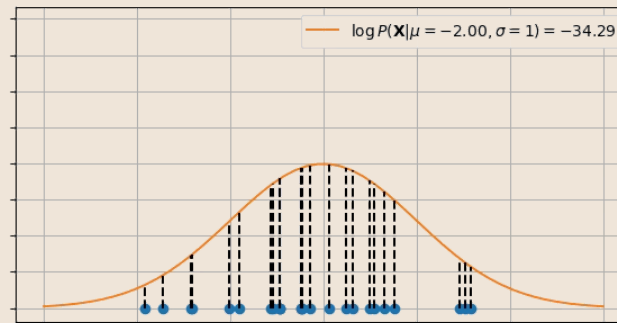
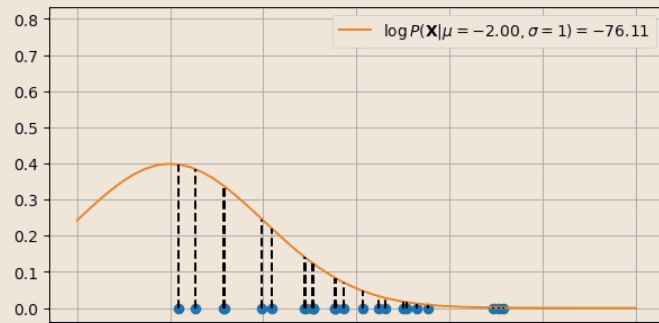
$$\Sigma_{ML} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \mu_{ML})(\mathbf{x}_i - \mu_{ML})^T$$

This measures how data varies around the mean in all directions.

The bottom line: "Fitting a Gaussian" is just calculating mean and covariance. What sounds like complex statistical inference reduces to basic descriptive statistics. The beauty lies in the theoretical justification: these estimates maximize the likelihood of observing our data.

How Does Optimization Work?

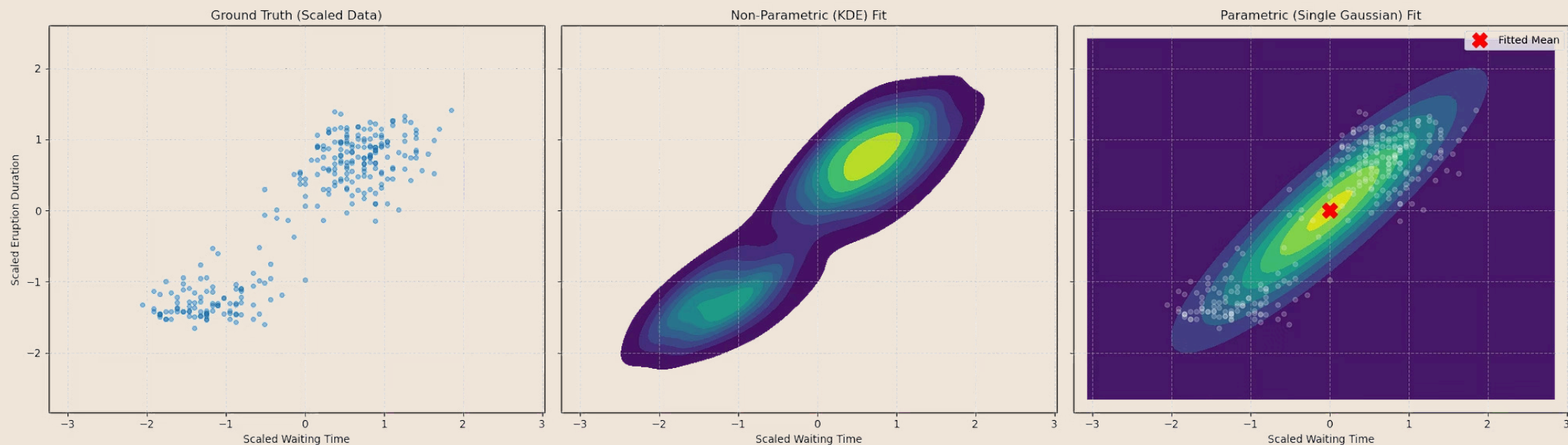
The plots below show the cost function (negative log-likelihood) for different choices of the parameters. While closed-form solutions exist in practice in the case of the Gaussian distribution, in other cases, one can use iterative optimization algorithms as in the case of Logistic Regressor.



When Single Gaussians Fail

Single Gaussian models assume **unimodal** data—one cluster. This fails for many real-world datasets with multiple distinct groups, like the Old Faithful geyser data with two clear clusters.

Model Bake-Off: Unimodal vs. Multimodal Data



Fitting a single Gaussian to bimodal data places the mean in the empty space *between* clusters—completely missing the true structure. Non-parametric KDE correctly finds both modes, but doesn't provide a compact mathematical model.

❏ **The Question:** Can we create a parametric model flexible enough to handle multi-modal data?

Gaussian Mixture Models (GMM)

Single Gaussian models struggle with multimodal data, as previously seen with the `geyser` dataset. While non-parametric KDE offers flexibility, it lacks a compact, mathematical model.

A **Gaussian Mixture Model (GMM)** provides a parametric solution by modeling data as a **weighted average (a "mixture")** of K different **Gaussian components**.

$$P(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$$

Mixing Coefficients (π_k)

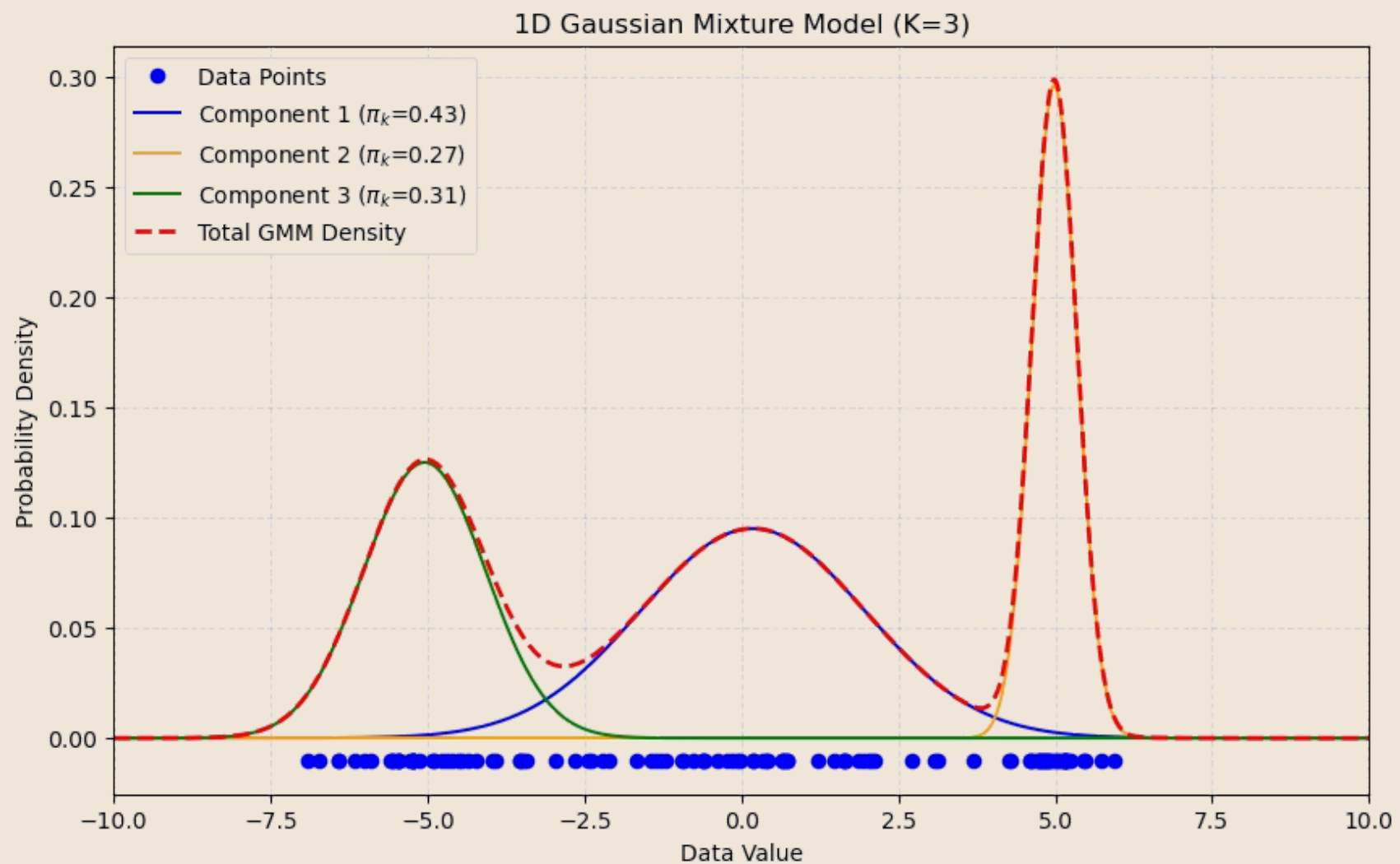
The "weight" or prior probability of each component. These must sum to 1.

Means (μ_k)

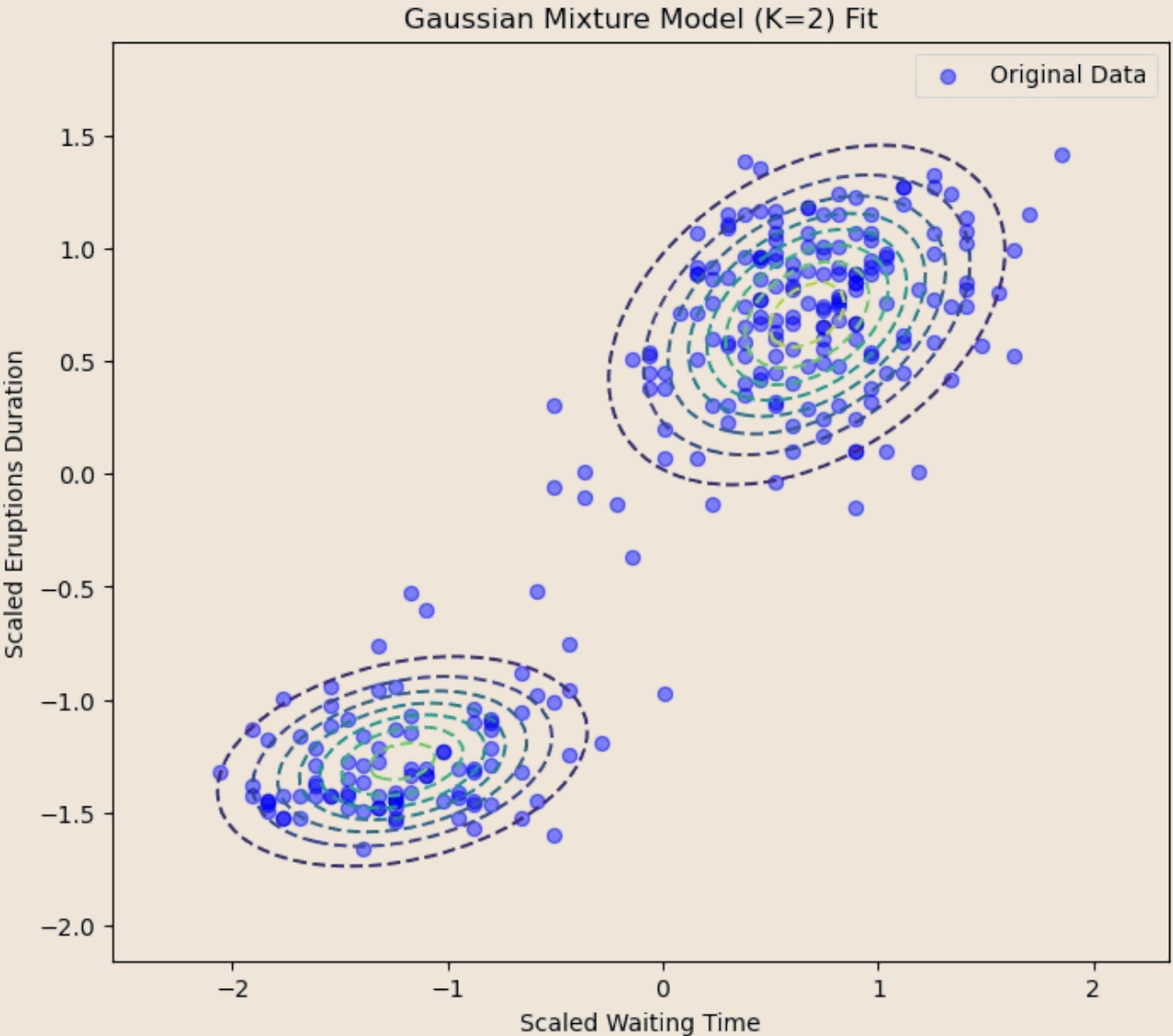
The center or location of each of the K Gaussian components.

Covariances (Σ_k)

The shape, spread, and orientation of each of the K components.



GMM Example on Geyser Data



GMM as "Soft Clustering"

GMM offers a nuanced alternative to K-Means' hard assignments. Instead of rigidly placing data points into clusters, GMM calculates the probability that a point belongs to each cluster, providing a **"soft" assignment**.

The Latent Variable Z

We introduce a **latent variable** (hidden variable) Z where $Z = k$ means an example $\mathbf{x}$ was generated by cluster k . We don't observe Z directly, but it's crucial for understanding cluster origin.

Computing Soft Assignments with Bayes' Theorem

To quantify this, we calculate the **posterior probability**, $P(Z = k|\mathbf{x})$, representing the likelihood of point $\mathbf{x}$ belonging to cluster k . This is achieved using **Bayes' Theorem**:

$$P(Z = k|\mathbf{x}) = \frac{P(\mathbf{x}|Z = k)P(Z = k)}{P(\mathbf{x})}$$

$P(\mathbf{x}|Z = k)$ (The Likelihood)

The probability of observing $\mathbf{x}$ if it came from cluster k . This is the value of the k -th Gaussian component: $\mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)$.

$P(Z = k)$ (The Prior)

How common is cluster k generally? This is our mixing coefficient, π_k .

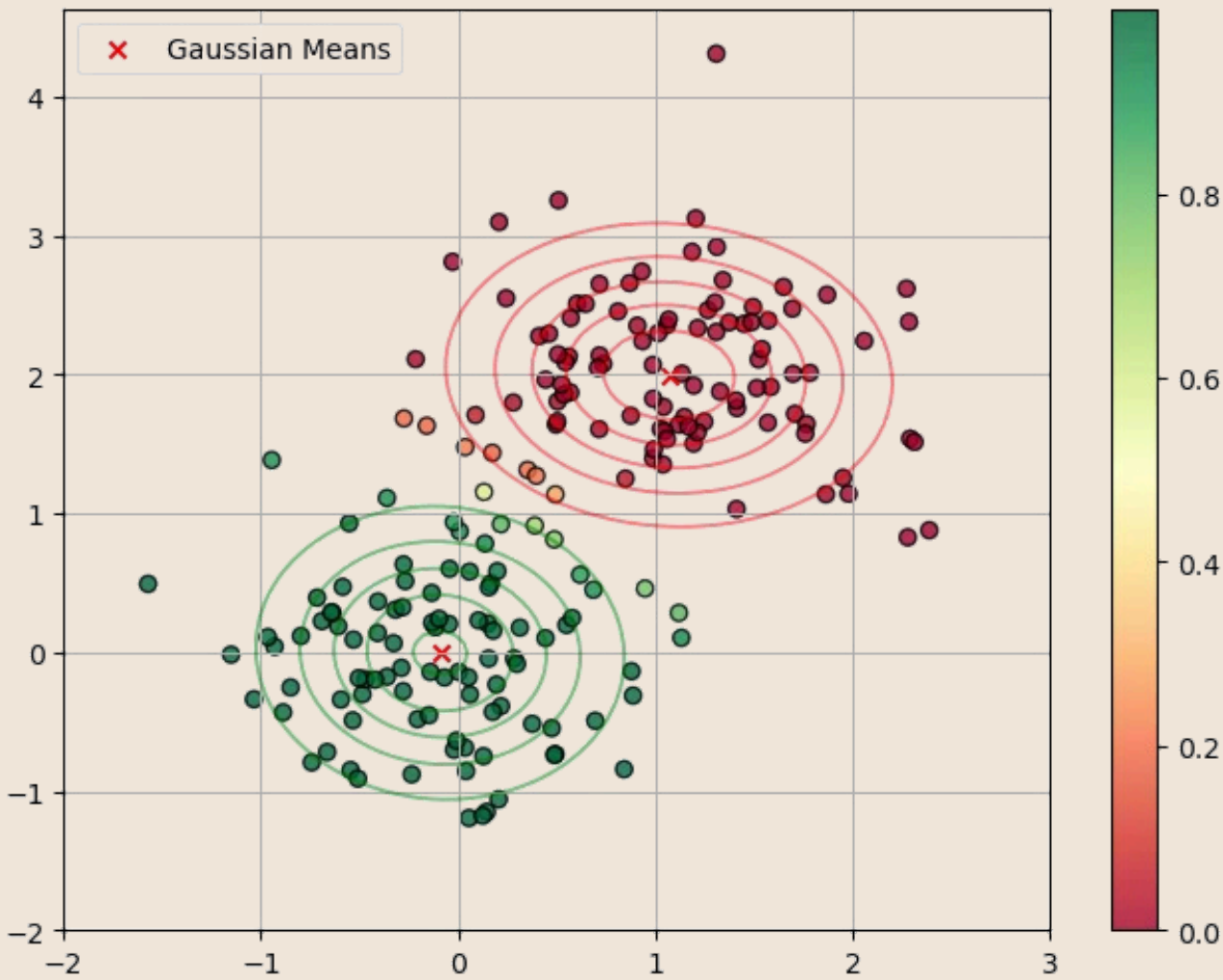
$P(\mathbf{x})$ (The Evidence)

The total probability of observing $\mathbf{x}$ across all clusters. This is the sum of all components: $\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}|\mu_j, \Sigma_j)$.

Combining these, the "soft assignment" (or **responsibility**, γ_k) for point $\mathbf{x}$ to cluster k is:

$$\gamma_k = P(Z = k|\mathbf{x}) = \frac{\pi_k \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}|\mu_j, \Sigma_j)}$$

The plot below shows points and assigned "responsibilities" coded by color:



How to "Learn" a GMM: Optimization by Maximum Likelihood

A Gaussian Mixture Model (GMM) is defined as a weighted sum of K Gaussian components:

$$P(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$$

To fully define the model, we need to determine its parameters: the mixing coefficients (π_k), means (μ_k), and covariances (Σ_k) for each of the K components.

Similar to fitting a single Gaussian, we use **Maximum Likelihood Estimation (MLE)**. The goal is to find the set of parameters $\theta = (\pi, \mu, \Sigma)$ that maximizes the log-likelihood of our observed data $\mathbf{X}$:

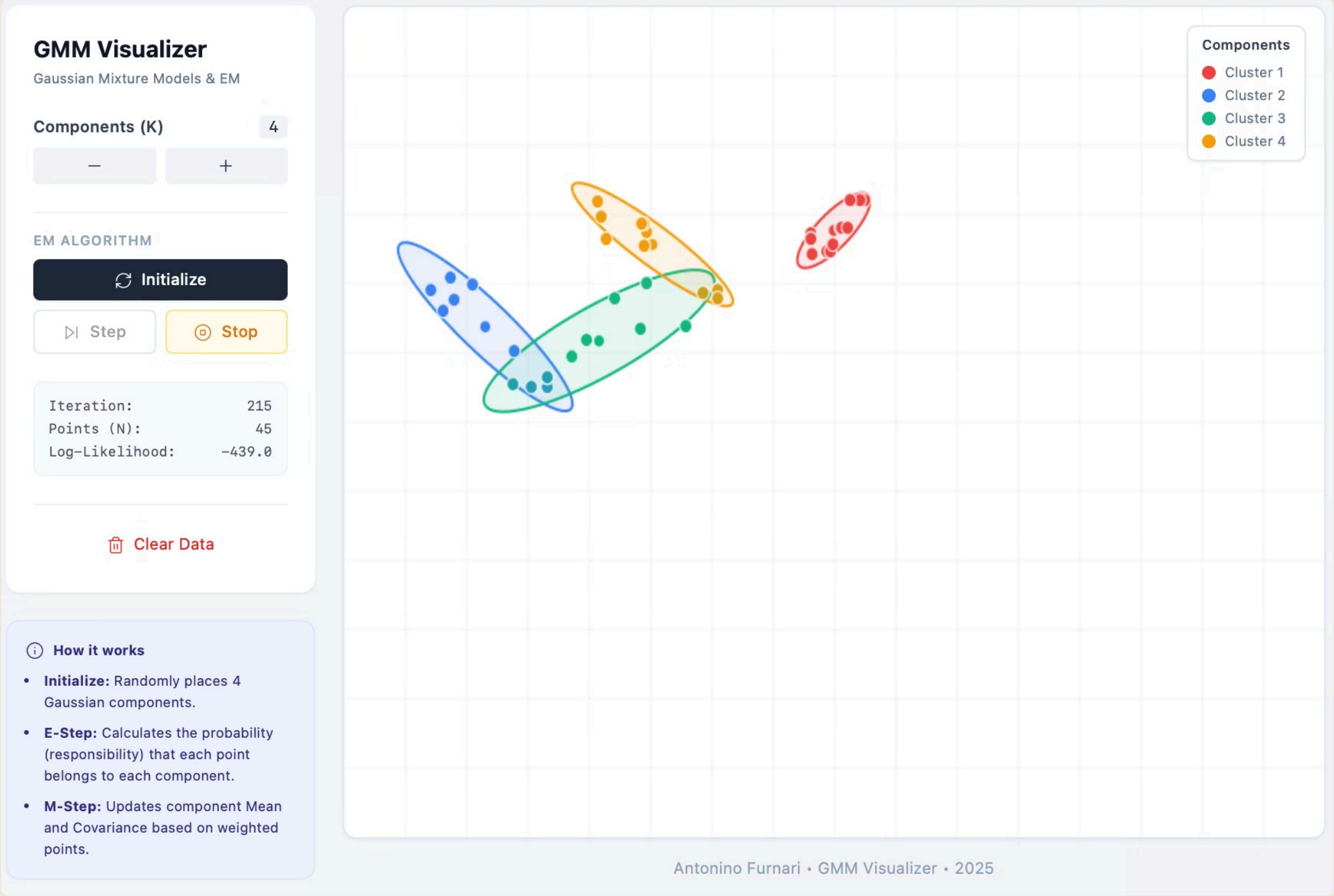
$$\log P(\mathbf{X} | \theta) = \sum_{i=1}^N \log \left[\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i | \mu_k, \Sigma_k) \right]$$

However, due to the sum inside the logarithm, this problem cannot be solved directly with a closed-form solution. Instead, a dedicated iterative algorithm called **Expectation-Maximization (EM)** is employed to find the optimal parameters.

The EM algorithm is similar in spirit to K-Means, but rather than just moving cluster centers, it iteratively adjusts means and covariance matrices.

Gaussian Mixture Models Demo

Gaussian Mixture Models Live Demo



GMM vs K-Means: A Closer Look

Both Gaussian Mixture Models (GMM) and K-Means are powerful clustering algorithms, but they operate on different assumptions and offer distinct approaches to grouping data. Understanding their core differences is crucial for selecting the right tool for your task.

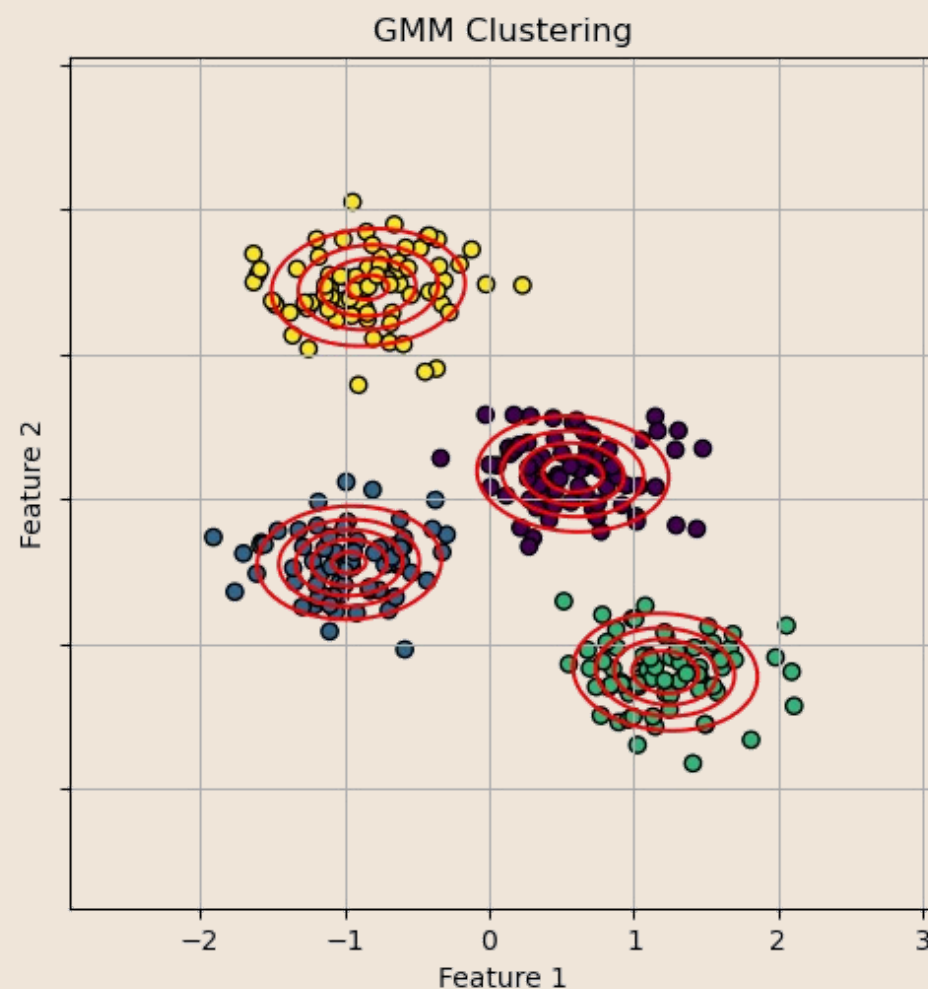
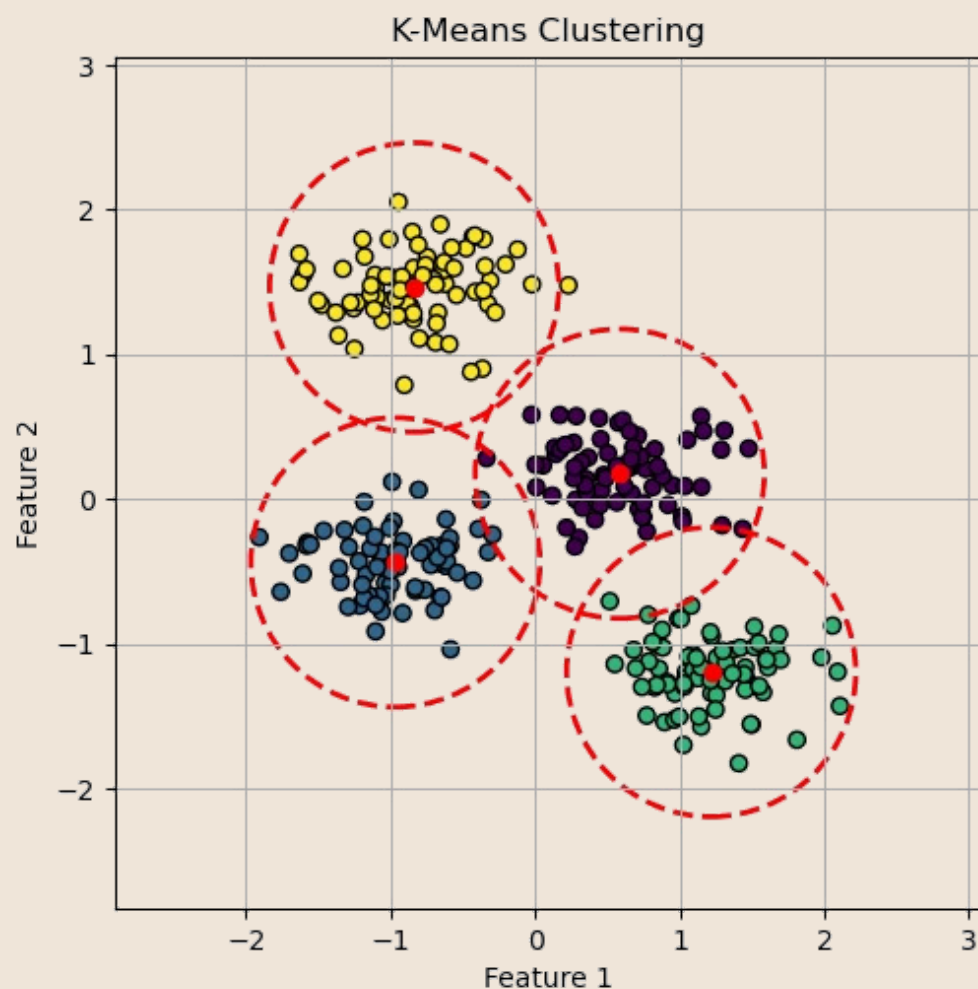
K-Means: Simpler, Faster

- **Hard Assignment:** Each data point is assigned exclusively to a single cluster. There's no ambiguity.
- **Spherical, Equal Covariance:** Assumes clusters are spherical, have the same size, and are aligned with the coordinate axes. This is often represented as diagonal covariance matrices $\epsilon \mathbf{I}$ where ϵ is shared across all components.
- **Efficiency:** Significantly faster convergence, making it a popular choice for large datasets when its assumptions hold.

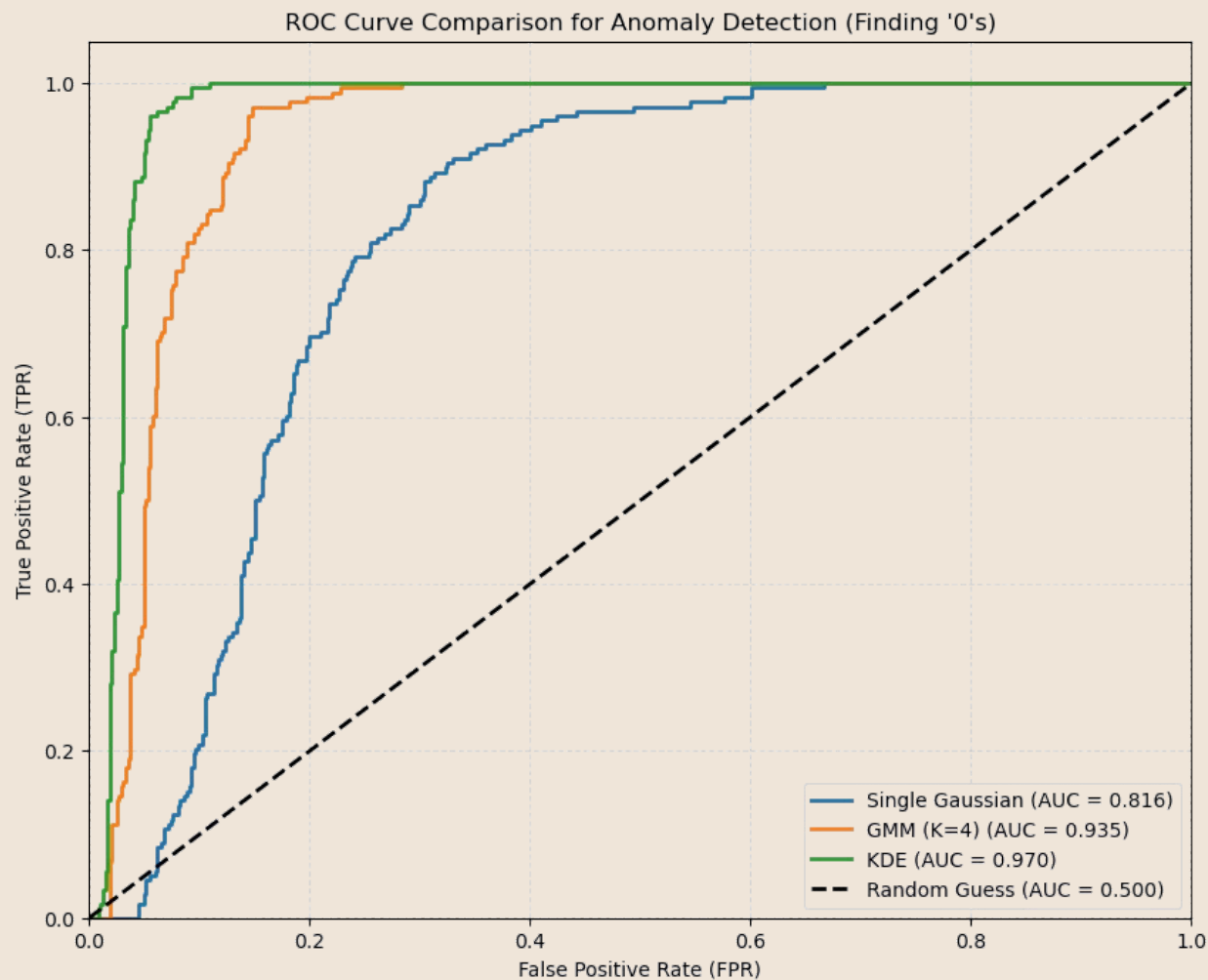
GMM: Flexible, Probabilistic

- **Soft Assignment (Responsibilities):** Data points have a probability (responsibility) of belonging to each cluster, allowing for nuanced relationships.
- **Flexible Covariance:** Each Gaussian component can have its own arbitrary covariance matrix, allowing for clusters of varying shapes, sizes, and orientations.
- **Density Estimation:** Beyond clustering, GMM provides a full probability density function, useful for understanding data generation.

While K-Means excels in speed and simplicity for data with well-separated, spherical clusters, GMM offers a more sophisticated, probabilistic approach capable of modeling complex, overlapping, and arbitrarily shaped distributions. The choice often boils down to the underlying data structure and whether a compact density model is required.



Lab: Density Estimation for Anomaly Detection



Conclusions and Next Steps



We Have Explored:

- Non-Parametric vs Parametric Density Estimation
- N-Dimensional Histograms
- Kernel Density Estimation
- Fitting Gaussians to the Data
- Gaussian Mixture Models

📅 In the next lectures, we will look at principal component analysis

References

- https://en.wikipedia.org/wiki/Multivariate_kernel_density_estimation
- https://en.wikipedia.org/wiki/Kernel_density_estimation
- http://faculty.washington.edu/yenchic/18W_425/Lec7_knn_basis.pdf
- Section 9.2 of [1]

[1] Bishop, Christopher M., and Nasser M. Nasrabadi. Pattern recognition and machine learning. Vol. 4. No. 4. New York: springer, 2006.